ARGUS · MARITIME EDGE VISION · MSDS CAPSTONE

Field Runbook

Quick-start deployment and command reference for the Argus script set

Rev 1.0 · August 2026 · Jetson Orin Nano (“NANO”) · L4T R39.2.0 / JetPack 7.2 · Arducam IMX585 B0498 + Canon EF 75–300 mm

Eight scripts, five stages. The ordering between stages is not cosmetic — several scripts produce output that *looks* correct but describes a configuration you did not actually use if they are run out of sequence. This runbook gives the flow, the commands, and the specific traps at each step.

Stage 0	Stage 1	Stage 2	Stage 3	Stage 4
Platform	**Preflight**	**Optics**	**Capture**	**Correlate**
once per flash	every session	every session	the session	workstation

1 The script set at a glance

Script	Stage	What it does	Exit codes
`argus_jetpack_check.sh`	0	Reports BSP (flash) and SDK (CUDA/TensorRT) state *separately*. A board can pass one and fail the other.	0 ok · 1 SDK absent · 2 BSP problem
`argus_camcheck.sh`	1	Finds the camera by VID:PID, confirms SuperSpeed, pins autosuspend off, resolves the capture node.	0 ready · 1 not ready
`argus_opticheck.sh`	2	Sets exposure/gain in the correct order, captures `.raw` frames, runs sweeps, appends DOPE rows.	0 ok · 1 capture/ measure failed
`argus_frame_stats.py`	2	Measurement engine. Exposure and Laplacian-focus stats on a headerless YUYV frame. Called by opticheck.	0 ok · 1 geometry · 2 truncated
`argus_raw2png.sh`	2	Renders those `.raw` frames as viewable PNGs so you can look at what you measured.	0 all ok · 1 some failed · 2 bad args
`capture_env.sh`	2	Provenance snapshot: version spine, power state, USB link, full camera capability table.	0 (read-only)
`argus-session.sh`	3	The production capture. Stills + clips + optional AIS, manifest, and frame index.	0 ok · 1 preflight failed · 2 bad args
`argus_correlate.py`	4	Joins AIS tracks against captured frames to produce geometric ground truth. Runs on the workstation.	0 ok · non-zero on missing inputs

DIRECTORY LAYOUT MATTERS

All eight files must sit in the same directory, and that directory should be a git repo. `argus_opticheck.sh` resolves `argus_frame_stats.py` relative to its own path; `argus-session.sh` resolves `argus_camcheck.sh` the same way; and `capture_env.sh` records the commit hash so a manifest can be traced back to the exact code that produced it.

2 Stage 0 — Platform setup (once per flash)

2.1 Put the scripts in place

```
cd ~/projects/computer-vision/maritime-edge
chmod +x argus_*.sh capture_env.sh argus-session.sh
git add -A && git commit -m "argus script set"
```

2.2 Confirm the SDK is actually installed

```
./argus_jetpack_check.sh --save        # writes manifests/jetpack-NANO-<UTC>.txt
echo $?                                # 0 = SDK operational
```

This script exists because the two install stages are easy to conflate. **Stage A** is the flashed BSP — kernel, bootloader, drivers. **Stage B** is the SDK — CUDA, cuDNN, TensorRT, VPI. A board can pass Stage A completely and have none of Stage B; the symptom is a working camera and a missing `nvcc`.

> **TWO OPEN WARNINGS ON THIS BOARD**
>
> Your last run reported `cuda_python not importable` and a torch warning that *no published PyTorch CUDA build for 2.13.0+cu132 supports this GPU* (compute capability 8.7) — while `torch.cuda.is_available()` still returns True. Resolve that pairing before any on-device latency figure goes into the capstone. A wheel without an `sm_87` kernel falls back to JIT or a mismatched arch, and the number you measure is not the number you would defend in a viva.

2.3 Create `site.conf` (needed only for AIS correlation)

```
cat > site.conf <<'EOF'
SITE_LAT=47.60000
SITE_LON=-122.34000
SITE_ALT_M=95
SITE_AZIMUTH_DEG=250
FOCAL_MM=300
EOF
```

`argus-session.sh` sources this into the session manifest; `argus_correlate.py` reads it back out. Skip it and capture still works — but correlation fails *after the fact*, with the footage already on disk and the vessels long gone. Add `site.conf` to `.gitignore` if you would rather not commit the rooftop coordinates.

3 Stage 1 — Preflight (every session)

```
./argus_camcheck.sh
```

EXPECTED OUTPUT

```
OK   link:  2-1.1 @ 5000M (SuperSpeed)
OK   node:  /dev/video0
READY
```

ORDER TRAP

Run this **before** `capture_env.sh`, never after. If the camera trained at 480M, the capability table that `capture_env.sh` records is the USB 2 table — 1280×720 caps, no 4K modes. You will have written a provenance file that confidently describes a configuration you did not use.

If it reports 480M: physically unplug and replug. Rebinding re-enumerates on the bus the device is already attached to, so it frequently cannot recover a link that failed to train as SuperSpeed in the first place. The serial descriptor `Arducam_20250915_0001` appears only at SuperSpeed, which makes it a reliable programmatic tell.

4 Stage 2 — Optics, exposure, and measurement

4.1 Declare the mechanical state first

V4L2 cannot see the lens barrel. Aperture, focus mark, and focal length are invisible to every control interface on the system, so they have to be supplied by hand — otherwise each DOPE row is attached to an unknown configuration and is worthless for comparison.

```
export ARGUS_LENS='canon-75-300' ARGUS_FOCAL=300 ARGUS_APERTURE=5.6 \
       ARGUS_FOCUS='infinity' ARGUS_RANGE=5000 ARGUS_COND='midday-clear' \
       SESSION=roof-01
export ARGUS_W=3840 ARGUS_H=2160 ARGUS_FPS=15
```

4.2 Look before you set

```
./argus_opticheck.sh status
```

Read three things off that report: the link speed, the `auto_exposure` state, and the exposure clamp ceiling. Exposure cannot exceed one frame period whatever value you write, so at 15 fps the ceiling is `exposure_time_absolute=666`. Values above it are silently clamped — you will not get an error, you will get a different exposure than the one in your log.

4.3 Set exposure and gain

```
./argus_opticheck.sh set 40 200        # 40 = 4 ms = 1/250 s shutter, gain 2.0x
```

CONTROL ORDERING IS LOAD-BEARING

`auto_exposure` must be set to Manual *before* `exposure_time_absolute` becomes writable — under AE it carries the `inactive` flag and writes are dropped without error. The script does this in the right order internally; the trap is only relevant if you drive `v4l2-ctl` by hand. Exposure values are in 100 µs units: 40 = 4 ms = 1/250 s.

4.4 Take a measured frame

```
./argus_opticheck.sh shot              # -> captures/shot_<UTC>.raw + DOPE row
```

4.5 Sweeps — when you need to find a setting, not confirm one

```
./argus_opticheck.sh sweep-exposure 100    # argument = the FIXED GAIN
./argus_opticheck.sh sweep-gain 40         # argument = the FIXED EXPOSURE
./argus_opticheck.sh sweep-focus           # interactive; exposure stays locked
```

> **THE ARGUMENTS ARE ASYMMETRIC**
>
> `sweep-exposure` takes a gain; `sweep-gain` takes an exposure. Passing 40 to `sweep-exposure` silently gives you a sweep at 0.4× gain rather than the 1/250 s shutter you meant. For rooftop work, pin `sweep-gain` at your real shutter (40), not the 200 default — shutter is fixed by vessel motion, so gain is the only free variable at dusk, and that sweep *is* your low-light budget.

4.6 Look at what you measured

```
./argus_raw2png.sh --crop-box              # captures/*.raw -> captures/*.png
```

This is the step that catches the gain-sweep trap. Laplacian variance rewards sensor noise, so a gain sweep can report rising "sharpness" across a series of frames that is visibly falling apart. `--crop-box` overlays the centred ROI the focus metric was computed on — if the target was not inside that box, the number describes water texture.

Flag	Use
`--all`	Extract every frame of a multi-frame file, not just index 0.
`--crop-box`	Draw the `ARGUS_CROP` ROI used by `argus_frame_stats.py`.
`--scale 1280x720`	Downscale for quick previews over SSH.
`--out DIR`	Write elsewhere instead of alongside the source.
`--range full`	Only if firmware changes the reported quantization from Limited.
`--force`	Overwrite existing PNGs (default is skip).

4.7 Snapshot provenance — last, not first

```
./capture_env.sh /dev/video0           # -> manifests/env-NANO-<UTC>.txt
```

Run this *after* the controls are set, so the manifest records the exposure and gain actually in force rather than the AE defaults the camera booted with. Every reproducibility field in your experiment log should point back at one of these files.

5 Stage 3 — Production capture

```
tmux new -s argus
./argus-session.sh --ais --shutter 40 --note "midday, clear, flood tide"

#   Ctrl-B D     detach (SSH can now drop safely)
#   tmux attach -t argus    reattach
#   Ctrl-C       stop cleanly
```

> **CTRL-C, NEVER KILL**
> Ctrl-C sends EOS so `splitmuxsink` finalises the last clip, and it is what triggers `frame_index.csv` construction and the closing manifest append. Killing the tmux window instead of interrupting the pipeline costs you the frame index — and without it, the AIS join in Stage 4 has nothing to join on.

Option	Effect
`--ais`	Co-launch AIS-catcher so both logs share one time window.
`--shutter N`	Exposure in 100 µs units. Pass the value you validated in Stage 2.
`--gain N`	Sensor gain; 100 = 1.0×.
`--stills-only`	Drop the clip branch. ~10 GB/hr instead of 60–125 GB/hr.
`--seg N`	Clip segment length in seconds (default 60).
`--note "..."`	Free text into the manifest. Conditions, tide, traffic.

The session re-runs `argus_camcheck.sh --quiet` itself, so Stage 1 is belt-and-braces here rather than strictly required. It sets exposure independently and inherits nothing from `argus_opticheck.sh`, so pass `--shutter` explicitly every time.

6 Stage 4 — Offload and correlate (workstation)

```
rsync -avh --progress argus.local:~/argus-data/20260805T183000Z/ ./20260805T183000Z/
python3 argus_correlate.py ./20260805T183000Z --max-range-km 15
```

Output is `ais_ground_truth.csv`: one row per frame-vessel pair with slant range, bearing offset, predicted pixel position, and predicted pixel length. That last column is what validates your optical model against real targets at known range.

> **AIS IS HIGH-PRECISION, LOW-RECALL**
> Vessels without transponders — most small craft, which are exactly the hard cases for your detector — are absent from this file but present in the imagery. Frames with no AIS hit are **not** negatives, and treating them as such will silently corrupt any recall figure you compute.

7 Quick start — first light in ten minutes

Cold board to a measured, viewable frame. Bench or rooftop.

```
cd ~/projects/computer-vision/maritime-edge

./argus_jetpack_check.sh           # 0 = SDK ok. Non-zero: stop and fix.
./argus_camcheck.sh                # 0 = SuperSpeed. 480M: replug, don't debug.

export ARGUS_LENS='canon-75-300' ARGUS_FOCAL=300 ARGUS_APERTURE=5.6 \
       ARGUS_FOCUS='infinity' ARGUS_COND='bench' SESSION=first-light
export ARGUS_W=1920 ARGUS_H=1080 ARGUS_FPS=15

./argus_opticheck.sh status        # read: link, auto_exposure, clamp ceiling
./argus_opticheck.sh set 40 200    # 1/250 s, 2.0x
```

```
./argus_opticheck.sh shot          # capture + measure + DOPE row
./argus_raw2png.sh --crop-box      # now go look at the PNG
```

Judge the frame against the targets below before changing anything. If the mean is far off, adjust gain first (shutter is pinned by motion), then re-shoot.

Metric	Target	Reading it
`luma mean`	90–140	Below: underexposed. Above: you are burning highlight headroom you need for glint.
`clipped`	< 1 %	Clipped highlights are unrecoverable. Sun glint will push this hard.
`crushed`	< 5 %	Crushed shadows erase dark hulls against dark water.
`lap_norm`	peak within a sweep	Valid *only* as a peak-finder inside one locked-down sweep. Never compare across sessions.

8 Copy-paste day sheet

```
# --- arrive on site, rig assembled, tethered ------------------------------
./argus_camcheck.sh || { echo "replug the camera"; exit 1; }

export ARGUS_LENS='canon-75-300' ARGUS_FOCAL=300 ARGUS_APERTURE=5.6 \
       ARGUS_FOCUS='infinity' ARGUS_RANGE=5000 ARGUS_COND='<conditions>' \
       SESSION='<session-id>'
export ARGUS_W=3840 ARGUS_H=2160 ARGUS_FPS=15

./argus_opticheck.sh status
./argus_opticheck.sh set 40 200
./argus_opticheck.sh shot
./argus_raw2png.sh --crop-box
./capture_env.sh

# --- production run -------------------------------------------------------
tmux new -s argus
./argus-session.sh --ais --shutter 40 --note "<conditions, tide, traffic>"
#   Ctrl-B D to detach.  Ctrl-C (not kill) to stop.

# --- before leaving -------------------------------------------------------
#   log the session in the DOPE book
#   confirm frame_index.csv exists and is non-empty

# --- back at the workstation ----------------------------------------------
rsync -avh --progress argus.local:~/argus-data/<STAMP>/ ./<STAMP>/
python3 argus_correlate.py ./<STAMP>
```

9 Environment variable reference

Variable	Default	Meaning
`ARGUS_DEV`	auto	Capture node. Auto-resolved by VID:PID; set to override.
`ARGUS_W` / `ARGUS_H`	1920 / 1080	Capture geometry. Must match what you pass to `argus_raw2png.sh`.
`ARGUS_FPS`	15	Frame rate. Sets the exposure clamp ceiling (10000/fps).

Variable	Default	Meaning
`ARGUS_SKIP`	9	Frames discarded before capture, letting AE/AWB settle.
`ARGUS_CROP`	600x400	Centred ROI for the focus metric and `--crop-box`.
`ARGUS_OUTDIR`	./captures	Where `.raw` frames land.
`ARGUS_DOPE`	./dope_book.csv	The measurement log.
`ARGUS_LENS`	16mm-stock	Lens identifier. **Change this** when the Canon is fitted.
`ARGUS_FOCAL`	16	Focal length in mm. **Change this** — 300 for the tele end.
`ARGUS_APERTURE`	unset	f-stop. Warns if left unset; row is unreproducible without it.
`ARGUS_FOCUS`	unset	Focus ring mark. Same warning.
`ARGUS_RANGE`	unset	Target range in metres, for stratifying results later.
`ARGUS_COND`	bench	Conditions string: light, weather, sea state.
`SESSION`	adhoc	Session ID written into every DOPE row.
`ARGUS_DATA`	~/argus-data	Root for `argus-session.sh` output.

> **THE DEFAULTS ARE BENCH DEFAULTS**
>
> `ARGUS_LENS=16mm-stock` and `ARGUS_FOCAL=16` describe the stock lens, not your rig. Forget to export them on the roof and every row in the DOPE book claims a 16 mm lens with a 38° field of view — internally consistent, entirely wrong, and impossible to detect from the file alone months later.

10 Troubleshooting

Symptom	Cause	Fix
Camera at 480M	SuperSpeed link failed to train	Physical replug. Rebinding rarely recovers it. Then try the bundled cable, then another USB 3 port.
SuperSpeed link but no capture node	`uvcvideo` not loaded, or only the metadata node matched	`sudo modprobe uvcvideo`; confirm with `v4l2-ctl --list-devices`.
`WARN: shutter did not take`	Value exceeded the fps clamp, or AE was still on	Check the ceiling in `opticheck status`; lower fps or exposure.
Only 4K@15 offered	YUYV-only firmware, ~2 Gbps FX3 ceiling	Expected, not a fault. 4K@30 uncompressed is physically impossible on this link.
PNG looks sheared or striped	Wrong `-W`/`-H` for that `.raw`	`argus_raw2png.sh` names the geometry the file size *is* consistent with — use it.
PNG looks flat and milky	Range expansion skipped	Default is `--range limited`. Do not pass `--range full` unless firmware changed.
“manifest is missing site geometry”	No `site.conf` at capture time	Edit `manifest.txt` by hand to rescue the session; create `site.conf` before the next.
AIS-catcher exits immediately	SDR not found, or contending with the camera	Check `ais/ais.log`. Keep the SDR on the USB 2 bus.

Symptom	Cause	Fix
Correlation finds no vessels	Azimuth wrong, or clock not NTP-synced	Verify `site_azimuth`; `timedatectl` must report `NTPSynchronized=yes`.
Engine build OOMs	8 GB unified memory	Swap is already at 8 GB. Also drop to `multi-user.target` to free the GUI's share.

11 Known gaps in the current set

Things that are not bugs but will bite if you assume otherwise. Worth resolving before the capture window closes.

- **Production frames are never measured.** `argus_frame_stats.py` and `argus_raw2png.sh` read headerless YUYV only. The session writes `.jpg` and `.mkv`, so nothing in the set checks exposure or focus on the footage you will actually train on.
- **White balance is locked on the bench but not in the session.** `argus_opticheck.sh` sets `white_balance_automatic=0`; `argus-session.sh` does not. Colour will drift across a long session in a way your bench characterisation does not predict.
- **The clipping thresholds may not match the sensor's range.** `argus_frame_stats.py` flags clipping at Y ≥ 250 and warns “headroom unused” below 200. If the ISP emits limited-range luma (16–235, as the driver reports), the clip check can never fire and the headroom warning fires on every correct exposure. One bright-target capture settles it; thresholds may need to become 235/190.
- **A stale docstring.** `argus_correlate.py` claims `pip install pyais`. It parses AIS-catcher JSON directly and imports nothing outside the standard library.
- **Frame timestamps come from mtime.** Adequate for AIS interpolation, which runs on 2–10 s reporting intervals, but do not build anything requiring tighter sync on top of it.